REVISION CHEAT SHEET:

SQL SERVER JOINS

Interview-ready notes • examples • SQL • diagrams

A **JOIN** combines rows from two or more table sources based on a relationship between them.

Suppose we have:

Customers

CustomerId	CustomerName
1	John
2	Priya
3	David

Orders

OrderId	CustomerId	Amount
101	1	500
102	1	700
103	2	300

The relationship is:

EXAMPLE / FLOW

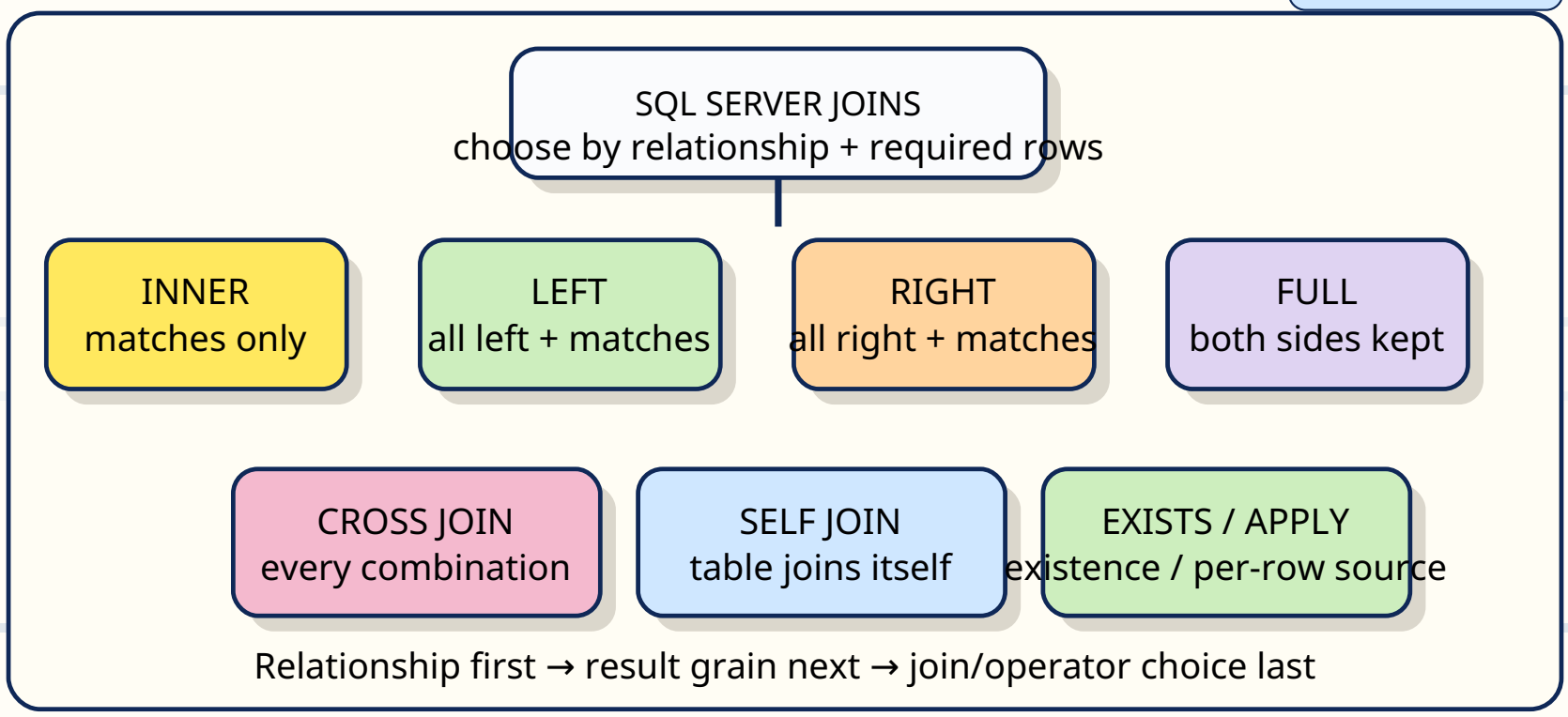
```
Customers.CustomerId
        ↓
Orders.CustomerId
```

A join allows us to return information from both tables:

EXAMPLE / FLOW

```
CustomerName + OrderId + Amount
```

SQL Server supports logical joins such as inner, left/right/full outer, and cross joins. The optimizer can physically execute joins using algorithms such as Nested Loops, Merge Join, Hash Join, and, where applicable, Adaptive Join.



Basic JOIN Syntax

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    o.OrderId,
    o.Amount
FROM dbo.Customers AS c
INNER JOIN dbo.Orders AS o
    ON o.CustomerId = c.CustomerId;
```

Important parts:

EXAMPLE / FLOW

```
dbo.Customers AS c
→ First table

dbo.Orders AS o
→ Second table

INNER JOIN
→ Type of join

ON o.CustomerId = c.CustomerId
→ Join condition
```

JOIN Condition Using ON

The `ON` clause tells SQL Server **how rows from the two table sources are related**.

Example:

SQL

```
ON o.CustomerId = c.CustomerId
```

This means:

Match an order with the customer whose `CustomerId` is the same.

Important SQL Server Rule: JOIN USING

Some databases support syntax such as:

```sql
JOIN Orders USING (CustomerId)
```

SQL Server T-SQL does **not** use the standard `JOIN ... USING (...)` form for normal joins.

Use:

```sql
JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
```

This is also clearer when the column names differ.

Table Aliases

Aliases make joins easier to read.

Instead of:

```sql
SELECT
    Customers.CustomerName,
    Orders.OrderId
FROM Customers
INNER JOIN Orders
    ON Orders.CustomerId = Customers.CustomerId;
```

use:

```sql
SELECT
    c.CustomerName,
    o.OrderId
FROM Customers AS c
INNER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId;
```

Here:

```
EXAMPLE / FLOW
c = Customers
o = Orders
```

Use meaningful aliases, especially when joining many tables.

J

INNER JOIN

Simple Definition

`INNER JOIN` returns only rows that have a match on both sides.

Think:

EXAMPLE / FLOW

```
Matching rows only
```

Example:

SQL

```
SELECT
    c.CustomerName,
    o.OrderId,
    o.Amount
FROM Customers AS c
INNER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId;
```

Result:

CustomerName	OrderId	Amount
John	101	500
John	102	700
Priya	103	300

David doesn't appear because David has no order.

INNER JOIN Diagram

EXAMPLE / FLOW

```
Customers
    ∩
Orders

Only matched rows
```

When to Use INNER JOIN

Use it when you only care about rows that have a valid match.

Examples:

EXAMPLE / FLOW

```
Orders that have customers

Order items that have products

Employees that belong to departments

Payments that belong to orders
```

When Not to Use INNER JOIN

Don't use it when rows from one side must still appear even when there is no match.

Example:

Show every customer, including customers who have never placed an order.

Use:

EXAMPLE / FLOW

```
LEFT JOIN
```

instead.

J

INNER JOIN Keyword

These are equivalent:

SQL

```
INNER JOIN
```

and:

SQL

```
JOIN
```

Example:

SQL

```
FROM Customers AS c
JOIN Orders AS o
    ON o.CustomerId = c.CustomerId;
```

`INNER` is optional.

For study notes, writing `INNER JOIN` explicitly can make the intent clearer.

J

LEFT JOIN / LEFT OUTER JOIN

Simple Definition

A `LEFT JOIN` returns:

EXAMPLE / FLOW

```
All rows from the left table
+
Matching rows from the right table
```

If there is no right-side match, SQL Server returns:

EXAMPLE / FLOW

```
NULL
```

for right-side columns.

`LEFT JOIN` and `LEFT OUTER JOIN` mean the same thing.

LEFT JOIN Example

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    o.OrderId
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId;
```

Result:

CustomerId	CustomerName	OrderId
1	John	101
1	John	102
2	Priya	103
3	David	NULL

David remains because `Customers` is the left table.

Easy LEFT JOIN Memory

EXAMPLE / FLOW

```
LEFT JOIN
=
Keep everything from LEFT
```

When to Use LEFT JOIN

Use it when the main table's rows must remain even without a related row.

Examples:

EXAMPLE / FLOW

```
All customers + their orders

All products + sales if any

All departments + employees if any

All users + last login if any
```

J

RIGHT JOIN / RIGHT OUTER JOIN

Simple Definition

A `RIGHT JOIN` returns:

EXAMPLE / FLOW

```
All rows from the right table
+
Matching rows from the left table
```

Unmatched left-side values become `NULL`.

`RIGHT JOIN` and `RIGHT OUTER JOIN` mean the same thing.

Example:

SQL

```sql
SELECT
    c.CustomerName,
    o.OrderId
FROM Orders AS o
RIGHT JOIN Customers AS c
    ON c.CustomerId = o.CustomerId;
```

This preserves all customers because `Customers` is on the right side.

LEFT JOIN vs RIGHT JOIN

These can often express the same logic simply by swapping table positions.

This:

SQL

```sql
Customers
LEFT JOIN Orders
```

is usually easier to read than:

SQL

```sql
Orders
RIGHT JOIN Customers
```

Many teams therefore use `LEFT JOIN` more often than `RIGHT JOIN`.

That is a readability preference, not a correctness rule.

J

FULL JOIN / FULL OUTER JOIN

Simple Definition

A `FULL OUTER JOIN` returns:

EXAMPLE / FLOW

```
Matches
+
Unmatched rows from the left
+
Unmatched rows from the right
```

`FULL JOIN` and `FULL OUTER JOIN` mean the same thing.

Example:

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    o.OrderId
FROM Customers AS c
FULL OUTER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId;
```

FULL OUTER JOIN Concept

EXAMPLE / FLOW

```
LEFT side unmatched
+
Matched
+
RIGHT side unmatched
```

Useful when comparing two datasets and you want to see:

EXAMPLE / FLOW

```
Records existing in both

Records only in A

Records only in B
```

J

CROSS JOIN

Simple Definition

A `CROSS JOIN` returns every possible combination of rows from both inputs.

This is called the:

Cartesian product

Microsoft describes `CROSS JOIN` as producing the cross-product of the two table sources.

Suppose:

Colors

EXAMPLE / FLOW

```
Red
Blue
```

Sizes

EXAMPLE / FLOW

```
Small
Medium
Large
```

Query:

SQL

```
SELECT
    c.ColorName,
    s.SizeName
FROM Colors AS c
CROSS JOIN Sizes AS s;
```

Result:

EXAMPLE / FLOW

```
Red   Small
Red   Medium
Red   Large

Blue  Small
Blue  Medium
Blue  Large
```

J

CROSS JOIN Row Count

If table A contains:

EXAMPLE / FLOW

```
100 rows
```

and table B:

EXAMPLE / FLOW

```
1,000 rows
```

then:

EXAMPLE / FLOW

```
100 × 1,000
=
100,000 rows
```

can be produced.

This is why accidental Cartesian products can be dangerous.

J

When to Use CROSS JOIN

Useful when every combination is actually required.

Examples:

EXAMPLE / FLOW

```
Every product × every store

Every date × every department

Every color × every size

Test-data generation
```

Avoiding Accidental Cartesian Products

Bad old-style query:

```sql
SELECT *
FROM Customers AS c,
     Orders AS o;
```

Without a matching condition, every customer combines with every order.

Prefer explicit joins:

```sql
SELECT *
FROM Customers AS c
INNER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId;
```

Explicit syntax makes the relationship much easier to inspect.

J

SELF JOIN

Simple Definition

A **self join** joins a table to itself.

This is useful when rows in the same table relate to other rows in that table.

Example:

Employees

EmployeeId	EmployeeName	ManagerId
1	Sarah	NULL
2	John	1
3	Priya	1

`ManagerId` references another:

EXAMPLE / FLOW

```
EmployeeId
```

in the same table.

J

SELF JOIN Example

```sql
SELECT
    e.EmployeeName AS Employee,
    m.EmployeeName AS Manager
FROM Employees AS e
LEFT JOIN Employees AS m
    ON m.EmployeeId = e.ManagerId;
```

Result:

Employee	Manager
Sarah	NULL
John	Sarah
Priya	Sarah

Aliases are essential here because both sources are the same table.

EXAMPLE / FLOW

```
e
→ Employee row

m
→ Manager row
```

Joining on Multiple Columns

A join condition can contain multiple comparisons.

Example:

SQL

```
SELECT *
FROM Sales AS s
INNER JOIN Targets AS t
    ON  t.RegionId = s.RegionId
    AND t.SalesYear = s.SalesYear;
```

Rows only match when:

EXAMPLE / FLOW

```
RegionId matches
AND
SalesYear matches
```

Joining Tables with Composite Keys

Suppose a table uses:

EXAMPLE / FLOW

```
OrderId + ProductId
```

as a composite key.

A related table may need to join using both columns.

Example:

SQL

```
SELECT *
FROM OrderItems AS oi
INNER JOIN ShipmentItems AS si
    ON  si.OrderId = oi.OrderId
    AND si.ProductId = oi.ProductId;
```

Important rule:

If the logical relationship is based on multiple key columns, join on the complete key.

Joining only one part can create incorrect matches.

Joining Three or More Tables

A query can contain multiple joins.

Suppose:

EXAMPLE / FLOW

```
Customers
    ↓
Orders
    ↓
OrderItems
    ↓
Products
```

Query:

SQL

```sql
SELECT
    c.CustomerName,
    o.OrderId,
    p.ProductName,
    oi.Quantity
FROM Customers AS c
INNER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
INNER JOIN OrderItems AS oi
    ON oi.OrderId = o.OrderId
INNER JOIN Products AS p
    ON p.ProductId = oi.ProductId;
```

Read it one relationship at a time:

EXAMPLE / FLOW

```
Customers → Orders

Orders → OrderItems

OrderItems → Products
```

Multiple JOIN Types in One Query

You can mix join types.

Example:

SQL

```sql
SELECT
    c.CustomerName,
    o.OrderId,
    p.PaymentId
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
LEFT JOIN Payments AS p
    ON p.OrderId = o.OrderId;
```

This can preserve customers even when they have:

EXAMPLE / FLOW

```
No order
```

or an order with:

EXAMPLE / FLOW

```
No payment
```

depending on the query structure.

Additional Conditions Inside ON

The `ON` condition can include more than the key relationship.

Example:

SQL

```sql
SELECT
    c.CustomerName,
    o.OrderId
FROM Customers AS c
LEFT JOIN Orders AS o
    ON  o.CustomerId = c.CustomerId
    AND o.Status = 'Open';
```

Meaning:

> Preserve every customer, but only match their open orders.

This differs importantly from putting the status filter in `WHERE`.

ON vs WHERE with INNER JOIN

For an `INNER JOIN`, these can often produce equivalent results:

SQL

```sql
FROM Customers AS c
INNER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
WHERE o.Status = 'Open';
```

versus:

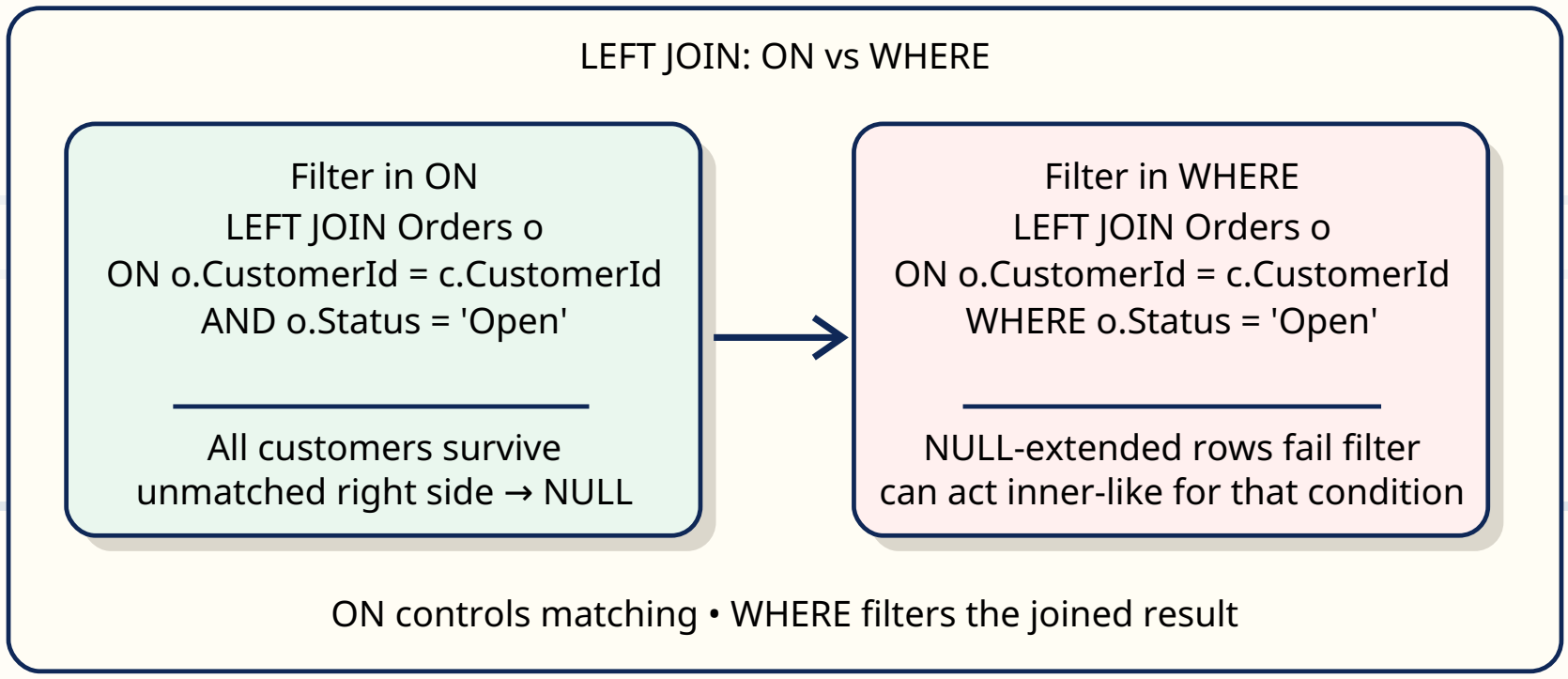
SQL

```sql
FROM Customers AS c
INNER JOIN Orders AS o
    ON  o.CustomerId = c.CustomerId
    AND o.Status = 'Open';
```

For inner joins, the distinction is often mainly logical/readability.

But for outer joins, placement can change the result.

ON vs WHERE with LEFT JOIN



This is extremely important.

Consider:

SQL

```
SELECT
    c.CustomerName,
    o.OrderId
FROM Customers AS c
LEFT JOIN Orders AS o
    ON  o.CustomerId = c.CustomerId
    AND o.Status = 'Open';
```

This means:

EXAMPLE / FLOW

```
Keep every customer

Match only Open orders
```

Customers without open orders still appear with:

EXAMPLE / FLOW

```
NULL
```

Moving the Filter to WHERE

SQL

```
SELECT
    c.CustomerName,
    o.OrderId
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
WHERE o.Status = 'Open';
```

Now rows where the right side is NULL fail the `WHERE` condition.

The result can effectively behave like an inner join for that condition.

Important Rule

For outer joins:

EXAMPLE / FLOW

```
ON
→ Controls which right-side rows qualify as matches

WHERE
→ Filters the result after the join
```

This difference can completely change results.

JOIN + WHERE

It is perfectly normal to join tables and then filter other conditions.

Example:

SQL

```sql
SELECT
    c.CustomerName,
    o.OrderId,
    o.TotalAmount
FROM Customers AS c
INNER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
WHERE
    o.TotalAmount >= 1000
    AND o.OrderDate >= '2026-01-01';
```

Here:

EXAMPLE / FLOW

```
ON
→ Relationship

WHERE
→ Required result filtering
```

Joining Tables from Different Schemas

Suppose:

EXAMPLE / FLOW

```
sales.Orders

crm.Customers
```

Query:

SQL

```sql
SELECT
    c.CustomerName,
    o.OrderId
FROM crm.Customers AS c
INNER JOIN sales.Orders AS o
    ON o.CustomerId = c.CustomerId;
```

Use:

EXAMPLE / FLOW

```
SchemaName.ObjectName
```

to identify the table clearly.

Joining Tables from Different Databases

Within the same SQL Server instance, a three-part object name can reference another database:

EXAMPLE / FLOW

```
Database.Schema.Object
```

Example:

SQL

```
SELECT
    c.CustomerName,
    o.OrderId
FROM CustomerDb.dbo.Customers AS c
INNER JOIN SalesDb.sales.Orders AS o
    ON o.CustomerId = c.CustomerId;
```

Required permissions and database accessibility must exist.

Four-Part Names

For linked-server scenarios, SQL Server can also use four-part naming:

EXAMPLE / FLOW

```
Server.Database.Schema.Object
```

Example conceptually:

EXAMPLE / FLOW

```
RemoteServer.SalesDb.sales.Orders
```

Linked-server design is a separate topic and can have significant security and performance implications.

JOIN with Aggregate Queries

Joins are commonly combined with aggregates.

Example:

Total order amount by customer.

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    COUNT(o.OrderId) AS OrderCount,
    SUM(o.TotalAmount) AS TotalSpent
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
GROUP BY
    c.CustomerId,
    c.CustomerName;
```

Important LEFT JOIN + COUNT Rule

After a left join, use:

SQL

```
COUNT(o.OrderId)
```

when counting actual orders.

Do not automatically use:

SQL

```
COUNT(*)
```

because a customer with zero orders still has an outer-join result row.

Handling SUM for Unmatched Rows

For a customer with no orders:

SQL

```
SUM(o.TotalAmount)
```

may return:

EXAMPLE / FLOW

```
NULL
```

You may deliberately display zero:

SQL

```
COALESCE(
    SUM(o.TotalAmount),
    0
) AS TotalSpent
```

only if zero correctly represents the business meaning.

JOIN with a Derived Table / Subquery

A subquery in the `FROM` clause can first summarize or transform data.

Example:

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    x.TotalSpent
FROM Customers AS c
LEFT JOIN
(
    SELECT
        CustomerId,
        SUM(TotalAmount) AS TotalSpent
    FROM Orders
    GROUP BY CustomerId
) AS x
    ON x.CustomerId = c.CustomerId;
```

The subquery first produces:

EXAMPLE / FLOW

```
One row per customer
```

then that result is joined to `Customers`.

Why Join to an Aggregated Subquery?

This can prevent duplicate multiplication.

Suppose a customer has many orders.

If you first aggregate to:

EXAMPLE / FLOW

```
CustomerId → one row
```

the outer query has a simpler relationship.

JOIN with a CTE

A CTE can make complex join logic easier to read.

Example:

SQL

```
WITH OrderTotals AS
(
    SELECT
        CustomerId,
        COUNT(*) AS OrderCount,
        SUM(TotalAmount) AS TotalSpent
    FROM Orders
    GROUP BY CustomerId
)
SELECT
    c.CustomerId,
    c.CustomerName,
    ot.OrderCount,
    ot.TotalSpent
FROM Customers AS c
LEFT JOIN OrderTotals AS ot
    ON ot.CustomerId = c.CustomerId;
```

The CTE is not necessarily a materialized temporary result. Think of it mainly as a named query expression; SQL Server's optimizer still determines the execution plan.

Handling Unmatched Rows

Outer joins are useful for finding unmatched data.

Example:

Show customers with no orders.

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
WHERE o.OrderId IS NULL;
```

Why This Works

A customer with no matching order produces:

EXAMPLE / FLOW

```
o.OrderId = NULL
```

Therefore:

SQL

```
WHERE o.OrderId IS NULL
```

selects only unmatched customers.

Finding Orphan Records

An orphan record is a child row whose parent doesn't exist.

A trusted foreign key normally prevents such invalid data.

But orphan data can occur in legacy systems, imported data, or systems without enforced constraints.

Example:

SQL

```
SELECT
    o.*
FROM Orders AS o
LEFT JOIN Customers AS c
    ON c.CustomerId = o.CustomerId
WHERE c.CustomerId IS NULL;
```

This identifies orders whose `CustomerId` doesn't match any customer.

J

INNER JOIN vs LEFT JOIN

Suppose:

EXAMPLE / FLOW

```
3 customers

Only 2 have orders
```

INNER JOIN:

EXAMPLE / FLOW

```
Only customers with orders
```

LEFT JOIN:

EXAMPLE / FLOW

```
All customers
+
orders where available
```

Quick comparison:

INNER JOIN	LEFT JOIN
Matches only	All left rows
Unmatched left rows disappear	Unmatched left rows remain
No NULL padding for missing match	Right columns become NULL
Good when match required	Good when match optional

Duplicate Rows Caused by JOINs

A join doesn't automatically produce one row per original row.

Suppose:

EXAMPLE / FLOW

```
Customer 1
```

has:

EXAMPLE / FLOW

```
3 orders
```

Then:

SQL

```
Customers
JOIN Orders
```

will naturally produce:

EXAMPLE / FLOW

```
3 rows
```

for that customer.

This is not a SQL Server error.

It reflects the relationship:

EXAMPLE / FLOW

```
1 customer
→ many orders
```

Many-to-Many Multiplication

Suppose an order has:

EXAMPLE / FLOW

```
3 OrderItems
```

and:

EXAMPLE / FLOW

```
2 Payments
```

Joining:

EXAMPLE / FLOW

```
Orders
→ OrderItems
→ Payments
```

may produce:

EXAMPLE / FLOW

```
3 × 2
=
6 rows
```

for one order.

If you then:

SQL

```
SUM(OrderAmount)
```

you may count the same value multiple times.

Fixing Duplicate Aggregation Problems

Do not immediately add:

SQL

```
DISTINCT
```

just to hide duplicates.

First understand the relationship.

Often the correct solution is to:

EXAMPLE / FLOW

```
Aggregate one child table first
```

Example:

SQL

```
WITH PaymentTotals AS
(
    SELECT
        OrderId,
        SUM(Amount) AS TotalPaid
    FROM Payments
    GROUP BY OrderId
)
SELECT
    o.OrderId,
    p.TotalPaid
FROM Orders AS o
LEFT JOIN PaymentTotals AS p
    ON p.OrderId = o.OrderId;
```

DISTINCT Is Not a Join Repair Tool

Bad habit:

SQL

```
SELECT DISTINCT ...
FROM A
JOIN B ...
JOIN C ...;
```

when the join itself is incorrect.

`DISTINCT` may:

EXAMPLE / FLOW

```
Hide duplicated output
```

without fixing:

EXAMPLE / FLOW

```
Incorrect relationships
Incorrect totals
Incorrect business logic
```

Join Cardinality

Before joining, understand whether the relationship is:

EXAMPLE / FLOW

```
One-to-one

One-to-many

Many-to-one

Many-to-many
```

This predicts how many rows the join may produce.

Example:

EXAMPLE / FLOW

```
Customers → Orders

1 → many
```

One customer can legitimately appear many times after joining to orders.

JOIN Performance Basics

Join performance depends on:

EXAMPLE / FLOW

```
Number of rows

Indexes

Join-column data types

Statistics

Data distribution

Filter selectivity

Join cardinality

Query shape
```

Index Join Columns

Columns frequently used for relationships often benefit from indexes.

Example:

EXAMPLE / FLOW

```
Customers.CustomerId
→ Primary key index

Orders.CustomerId
→ Often useful child-side index
```

A foreign key does not automatically mean SQL Server created an index on the child column.

Matching Data Types

Prefer join columns with compatible data types.

Good:

EXAMPLE / FLOW

```
Customers.CustomerId INT

Orders.CustomerId INT
```

Poor design:

EXAMPLE / FLOW

```
Customers.CustomerId INT

Orders.CustomerId VARCHAR(20)
```

Joining different data types can require conversions and may hurt performance or produce conversion problems.

Avoid Functions on Join Columns When Possible

Less ideal:

SQL

```
ON YEAR(a.OrderDate) = YEAR(b.TargetDate)
```

or:

SQL

```
ON UPPER(a.Code) = UPPER(b.Code)
```

Such expressions can make index usage harder.

Design consistent, directly comparable keys when possible.

Filter Early, but Preserve Correct Semantics

Filtering rows can reduce work.

Example:

SQL

```
WHERE OrderDate >= '2026-01-01'
  AND OrderDate <  '2027-01-01'
```

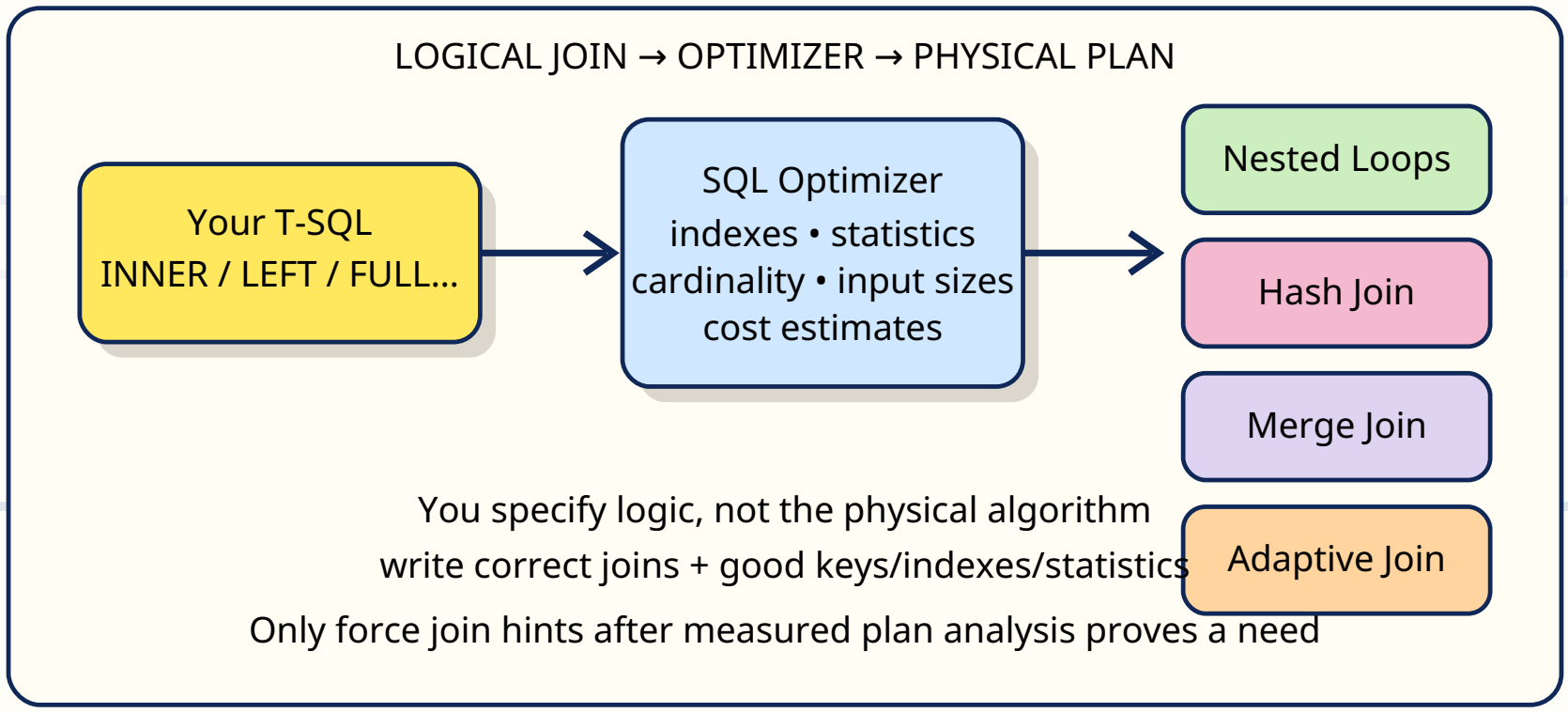
can substantially reduce rows entering later operations.

But do not move filters between `ON` and `WHERE` blindly because outer-join semantics can change.

Correctness comes first.

J

Logical Join vs Physical Join



When you write:

```sql
INNER JOIN
```

you describe the **logical relationship** you want.

SQL Server's optimizer then chooses a physical algorithm, such as:

EXAMPLE / FLOW

```
Nested Loops

Hash Join

Merge Join

Adaptive Join
```

based on available indexes, cardinality estimates, input sizes, and other plan considerations.

Nested Loops — Basic Idea

Conceptually:

EXAMPLE / FLOW

```
For each row from input A
    find matching rows in B
```

Often effective when:

EXAMPLE / FLOW

```
Outer input is relatively small
+
Inner side can be efficiently searched
```

The optimizer makes the decision; you normally don't manually choose it.

Hash Join — Basic Idea

SQL Server can build a hash structure from one input and probe it with the other.

Often useful for:

EXAMPLE / FLOW

```
Larger unsorted inputs

Equality joins

Cases without ideal supporting indexes
```

Again, this is optimizer behavior, not something you normally specify manually.

Merge Join — Basic Idea

Merge Join works efficiently when both inputs are appropriately ordered on the join keys.

Microsoft notes that Merge Join requires sorted join inputs and can be especially effective when suitable indexes already provide the needed ordering.

Do Not Force Join Algorithms Casually

SQL Server supports join hints, but they restrict optimizer choices.

Microsoft warns that forcing join behavior can prevent the optimizer from selecting a better plan and can even make a query slower.

Rule:

> Start by writing correct relational SQL and maintaining good indexes/statistics. Force a physical join only after measurement and plan analysis demonstrate a real need.

J

CROSS APPLY

`APPLY` is an important SQL Server-specific query operator.

Simple Definition

`CROSS APPLY` evaluates the right-side table expression for **each row on the left**.

Microsoft documents that `APPLY` evaluates the right table source against every row from the left source. With `CROSS APPLY`, a left-side row disappears when the right side produces no row.

Think:

EXAMPLE / FLOW

```
For each row on the left
→ Run/evaluate this right-side table expression
```

J

CROSS APPLY with a Table-Valued Function

Suppose:

EXAMPLE / FLOW

```
dbo.GetRecentOrders(CustomerId)
```

returns orders for one customer.

Query:

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    x.OrderId
FROM Customers AS c
CROSS APPLY dbo.GetRecentOrders(c.CustomerId) AS x;
```

The function receives a value from the current customer row.

This correlated table-source behavior is one of the main reasons `APPLY` exists. Microsoft specifically documents table-valued-function scenarios for `CROSS APPLY`.

J

CROSS APPLY with a Correlated Subquery

A very useful pattern:

Latest order for each customer.

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    x.OrderId,
    x.OrderDate
FROM Customers AS c
CROSS APPLY
(
    SELECT TOP (1)
        o.OrderId,
        o.OrderDate
    FROM Orders AS o
    WHERE o.CustomerId = c.CustomerId
    ORDER BY o.OrderDate DESC
) AS x;
```

For each customer:

EXAMPLE / FLOW

```
Find that customer's latest order
```

J

CROSS APPLY Behavior

If a customer has no orders:

EXAMPLE / FLOW

```
Right side returns nothing
```

Therefore:

EXAMPLE / FLOW

```
Customer disappears
```

This resembles inner-join behavior.

OUTER APPLY

`OUTER APPLY` works similarly, except it preserves left-side rows when the right-side expression returns nothing.

Microsoft documents that `OUTER APPLY` returns a left-side row even when the right-side table expression produces an empty result, filling the right-side columns with NULL.

OUTER APPLY Example

SQL

```
SELECT
    c.CustomerId,
    c.CustomerName,
    x.OrderId,
    x.OrderDate
FROM Customers AS c
OUTER APPLY
(
    SELECT TOP (1)
        o.OrderId,
        o.OrderDate
    FROM Orders AS o
    WHERE o.CustomerId = c.CustomerId
    ORDER BY o.OrderDate DESC
) AS x;
```

Now customers with no order remain:

EXAMPLE / FLOW

```
CustomerName   OrderId
----------------------
David          NULL
```

CROSS APPLY vs OUTER APPLY

EXAMPLE / FLOW

```
CROSS APPLY
→ Keep left row only when right side returns rows

OUTER APPLY
→ Keep every left row
```

Memory:

EXAMPLE / FLOW

```
CROSS APPLY
≈ Inner-like behavior

OUTER APPLY
≈ Left-outer-like behavior
```

But `APPLY` is especially valuable because the right-side expression can depend directly on columns from the current left row.

JOIN vs APPLY

Use normal `JOIN` when two rowsets can be related with an ordinary join predicate.

Use `APPLY` particularly when:

EXAMPLE / FLOW

```
Right-side table expression depends on each left row

Calling a table-valued function with a left-row value

Top N related rows per parent

Correlated table expression is clearer
```

EXISTS — Semi Join Pattern

Sometimes you do not need columns from the related table.

You only need to know:

> Does at least one matching row exist?

Use:

SQL

```
EXISTS
```

Microsoft defines `EXISTS` as testing whether its subquery produces rows.

EXISTS Example

Find customers who have at least one order:

SQL

```sql
SELECT
    c.CustomerId,
    c.CustomerName
FROM Customers AS c
WHERE EXISTS
(
    SELECT 1
    FROM Orders AS o
    WHERE o.CustomerId = c.CustomerId
);
```

Why SELECT 1?

Inside `EXISTS`, SQL Server only cares whether a row exists.

The actual selected value is not important.

Therefore:

SQL

```sql
SELECT 1
```

clearly expresses:

EXAMPLE / FLOW

```
Only existence matters
```

J

Semi Join Concept

Conceptually, `EXISTS` is a **semi-join pattern**:

EXAMPLE / FLOW

```
Return row from left
if a match exists on right
```

Unlike an ordinary inner join, it does not need to return right-side columns or duplicate the left row for every matching child.

SQL Server execution plans include logical Left Semi Join and Right Semi Join operators for existence-style operations.

JOIN vs EXISTS

Suppose a customer has 10 orders.

This:

SQL

```sql
SELECT
    c.CustomerId
FROM Customers AS c
INNER JOIN Orders AS o
    ON o.CustomerId = c.CustomerId;
```

can return the customer's ID:

EXAMPLE / FLOW

```
10 times
```

unless you add grouping or `DISTINCT`.

But:

SQL

```sql
SELECT
    c.CustomerId
FROM Customers AS c
WHERE EXISTS
(
    SELECT 1
    FROM Orders AS o
    WHERE o.CustomerId = c.CustomerId
);
```

returns the customer once.

When to Prefer EXISTS

Use `EXISTS` when the question is:

EXAMPLE / FLOW

```
Does a related row exist?
```

Examples:

EXAMPLE / FLOW

```
Customers who have orders

Products that have sales

Employees with active assignments
```

Use a join when you actually need columns from both sides.

J NOT EXISTS — Anti Join Pattern

`NOT EXISTS` means:

Return a row when no matching related row exists.

Example:

SQL

```sql
SELECT
    c.CustomerId,
    c.CustomerName
FROM Customers AS c
WHERE NOT EXISTS
(
    SELECT 1
    FROM Orders AS o
    WHERE o.CustomerId = c.CustomerId
);
```

This returns customers with no orders.

J Anti Join Concept

Conceptually:

EXAMPLE / FLOW

```
Left row
+
No matching right row
=
Return it
```

SQL Server execution plans include logical Left Anti Semi Join and Right Anti Semi Join operators for these patterns.

NOT EXISTS vs LEFT JOIN IS NULL

These often express the same business requirement.

LEFT JOIN pattern

SQL

```
SELECT
    c.CustomerId
FROM Customers AS c
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
WHERE o.OrderId IS NULL;
```

NOT EXISTS pattern

SQL

```
SELECT
    c.CustomerId
FROM Customers AS c
WHERE NOT EXISTS
(
    SELECT 1
    FROM Orders AS o
    WHERE o.CustomerId = c.CustomerId
);
```

Both can mean:

EXAMPLE / FLOW

```
Customers with no orders
```

`NOT EXISTS` often expresses the anti-match intent more directly.

NOT EXISTS vs NOT IN

Be especially careful with:

SQL

```
NOT IN
```

when the subquery can contain `NULL`.

Example:

SQL

```
WHERE CustomerId NOT IN
(
    SELECT CustomerId
    FROM Orders
)
```

A `NULL` in the subquery can produce unexpected three-valued-logic behavior.

For correlated anti-match logic, this is often safer:

SQL

```
WHERE NOT EXISTS (...)
```

JOIN Elimination — Basic Optimizer Behavior

Sometimes your query mentions a table whose presence is not actually necessary to calculate the result.

Example conceptually:

SQL

```
SELECT
    o.OrderId
FROM Orders AS o
INNER JOIN Customers AS c
    ON c.CustomerId = o.CustomerId;
```

If SQL Server can prove from trusted constraints that every valid order must have a corresponding customer and no customer column is otherwise needed, the optimizer may be able to simplify the plan.

Trusted Foreign Keys Matter

Microsoft documents that trusted foreign-key constraints provide facts the optimizer can use when simplifying execution plans. If a foreign key is disabled or marked not trusted, the optimizer cannot safely rely on the existing data satisfying that relationship.

This is one reason properly maintained constraints provide more than just data validation.

Important Join Elimination Rule

Do not write unnecessary joins expecting:

EXAMPLE / FLOW

```
"The optimizer will always remove them."
```

Join elimination depends on:

EXAMPLE / FLOW

```
Query shape

Constraint information

Constraint trust

Selected columns

Join type

Optimizer decisions
```

Write clear, necessary joins first.

Treat optimizer simplification as an optimization opportunity, not a substitute for clean SQL.

Optimizer Can Reorder Joins

If you write:

EXAMPLE / FLOW

```
A JOIN B JOIN C
```

SQL Server is not necessarily required to physically execute them in exactly that written order.

The optimizer evaluates possible plans and chooses an execution strategy based on cost estimates.

This is why logical SQL and physical execution are different concepts.

Important Join Rules

Remember these rules:

EXAMPLE / FLOW

1. Join on the real relationship.
2. Use all columns of a composite relationship.
3. Understand one-to-one / one-to-many / many-to-many.
4. Use INNER JOIN when a match is required.
5. Use LEFT JOIN when left rows must survive.
6. Put outer-join match restrictions carefully in ON.
7. Use WHERE for final row filtering.
8. Avoid accidental CROSS JOINs.
9. Do not use DISTINCT to hide incorrect joins.
10. Index important join columns appropriately.
11. Keep join-column data types compatible.
12. Use EXISTS when only existence matters.
13. Use NOT EXISTS when absence matters.
14. Use APPLY for correlated table expressions or TVFs.
15. Let the optimizer choose physical join algorithms unless measured evidence says otherwise.

Core JOIN Cheat Sheet

Join	Returns
`INNER JOIN`	Matching rows only
`LEFT JOIN`	All left + matched right
`RIGHT JOIN`	All right + matched left
`FULL OUTER JOIN`	All matches + unmatched from both
`CROSS JOIN`	Every combination
Self Join	Table joined to itself
`CROSS APPLY`	Left rows whose per-row right expression returns rows
`OUTER APPLY`	Every left row + per-row right result if any

Matching / Existence Cheat Sheet

Requirement	Common Pattern
Need columns from both tables	`JOIN`
Need to know whether match exists	`EXISTS`
Need rows with no match	`NOT EXISTS`

Requirement	Common Pattern
Need all left rows	`LEFT JOIN`
Need unmatched left rows	`LEFT JOIN ... IS NULL` or `NOT EXISTS`
Need correlated table expression	`APPLY`
Need every combination	`CROSS JOIN`

J

Complete Practical Example

Suppose we have:

EXAMPLE / FLOW

```
crm.Customers
sales.Orders
sales.OrderItems
catalog.Products
billing.Payments
```

We want:

For every active customer, show their order, products, and total payment information where available.

SQL

```
WITH PaymentTotals AS
(
    SELECT
        OrderId,
        SUM(Amount) AS TotalPaid
    FROM billing.Payments
    GROUP BY OrderId
)
SELECT
    c.CustomerId,
    c.CustomerName,
    o.OrderId,
    o.OrderDate,
    p.ProductId,
    p.ProductName,
    oi.Quantity,
    oi.UnitPrice,
    pt.TotalPaid
FROM crm.Customers AS c
LEFT JOIN sales.Orders AS o
    ON o.CustomerId = c.CustomerId
LEFT JOIN sales.OrderItems AS oi
    ON oi.OrderId = o.OrderId
LEFT JOIN catalog.Products AS p
    ON p.ProductId = oi.ProductId
LEFT JOIN PaymentTotals AS pt
    ON pt.OrderId = o.OrderId
WHERE c.IsActive = 1;
```

Why aggregate payments first?

Because if one order contains:

EXAMPLE / FLOW

```
3 items
2 payments
```

joining raw items directly to raw payments can multiply rows.

The CTE converts payments into:

EXAMPLE / FLOW

```
One payment-summary row per order
```

before joining.

Common Join Mistakes

Wrong Join Column

Bad:

SQL

```
ON Orders.OrderId = Customers.CustomerId
```

just because both columns happen to be integers.

Join columns must represent the real relationship.

Missing Part of Composite Key

If the relationship is:

EXAMPLE / FLOW

```
OrderId + ProductId
```

don't join only:

SQL

```
ON a.OrderId = b.OrderId
```

unless that is truly sufficient.

LEFT JOIN Filter in WHERE

Potential mistake:

SQL

```
LEFT JOIN Orders AS o
    ON o.CustomerId = c.CustomerId
WHERE o.Status = 'Open';
```

This removes customers without an order.

If you need all customers:

SQL

```
LEFT JOIN Orders AS o
    ON  o.CustomerId = c.CustomerId
    AND o.Status = 'Open';
```

Accidental Cartesian Product

Forgetting a join condition can multiply rows dramatically.

Adding DISTINCT Instead of Fixing Cardinality

`DISTINCT` removes duplicate output values. It does not necessarily repair an incorrect join.

Joining Many Child Tables Before Aggregating

This can cause totals to multiply.

Aggregate each one-to-many source to the desired grain when appropriate.

Using INNER JOIN When Unmatched Rows Matter

If the requirement says:

EXAMPLE / FLOW

```
"all customers"
```

an inner join may accidentally remove customers with no activity.

Using JOIN When EXISTS Is Enough

If you need only:

EXAMPLE / FLOW

```
Customers that have orders
```

`EXISTS` may communicate the intent better than joining orders and removing duplicates afterward.

J

30 Interview Questions

1. What is a JOIN?

A join combines rows from two or more table sources based on a relationship or matching condition.

2. What is an INNER JOIN?

An inner join returns only rows that satisfy the join condition on both sides. Unmatched rows are removed.

3. What is a LEFT JOIN?

A left join returns every row from the left table and matching rows from the right table. Missing right-side matches appear as NULL.

4. What is a RIGHT JOIN?

A right join preserves every row from the right table and matching rows from the left. It can often be rewritten as a left join by reversing table order.

5. What is a FULL OUTER JOIN?

It returns matched rows plus unmatched rows from both inputs, using NULL for missing values on the opposite side.

6. What is a CROSS JOIN?

A cross join returns the Cartesian product: every row from one source combined with every row from the other.

7. What is a self join?

A self join joins a table to itself, commonly for hierarchical relationships such as employee-to-manager.

8. What does the ON clause do?

`ON` defines the relationship used to decide which rows from the joined table sources match.

9. Does SQL Server support JOIN USING?

Normal SQL Server T-SQL joins use `ON`; the `JOIN ... USING(column)` syntax found in some other database systems isn't the standard SQL Server join syntax.

10. Can a join use multiple columns?

Yes. Use multiple conditions joined with `AND`, especially for relationships based on composite keys.

11. How do you join three or more tables?

Add multiple join clauses, with each `ON` expression representing the appropriate relationship between table sources.

12. What is the difference between ON and WHERE?

`ON` determines join matching. `WHERE` filters the resulting rows. With outer joins, moving a condition between them can change results.

13. Why can a LEFT JOIN behave like an INNER JOIN?

A `WHERE` condition requiring a right-table column value can reject the NULL rows created for unmatched left rows.

14. How do you find rows with no matching child record?

Use `LEFT JOIN ... WHERE child.Key IS NULL` or, often more directly, `NOT EXISTS`.

15. What is an orphan record?

An orphan is a child row whose referenced parent doesn't exist. Proper trusted foreign-key constraints normally prevent this condition.

16. Why do joins create duplicate-looking rows?

A one-to-many relationship naturally returns the parent once for each matching child. The apparent duplicates reflect relationship cardinality.

17. Why can joins make aggregate totals wrong?

Joining multiple one-to-many child tables can multiply rows, causing values to be counted several times. Aggregate child sources first when necessary.

18. Should DISTINCT be used to fix duplicate rows from joins?

Not automatically. First determine whether the duplicates are legitimate or caused by incorrect join logic. `DISTINCT` can hide a design/query error.

19. How do you join tables from different schemas?

Use schema-qualified names, such as `crm.Customers` and `sales.Orders`.

20. How do you join tables from different databases?

On the same SQL Server instance, use three-part names such as `CustomerDb.dbo.Customers` and `SalesDb.sales.Orders`, assuming permissions allow access.

21. What is CROSS APPLY?

`CROSS APPLY` evaluates a right-side table expression for each row from the left side and excludes left rows when that expression produces no result.

22. What is OUTER APPLY?

`OUTER APPLY` also evaluates the right-side expression per left row but preserves the left row even when the right side returns nothing.

23. When is APPLY useful?

It is especially useful with correlated table expressions, table-valued functions, and patterns such as retrieving the top related row for each parent.

24. What does EXISTS do?

`EXISTS` checks whether its subquery returns at least one row. It is ideal when only the existence of a related row matters.

25. What is a semi join?

Conceptually, a semi join returns a left-side row when a matching right-side row exists, without returning the right-side rows themselves. `EXISTS` commonly represents this pattern.

26. What is an anti join?

An anti join returns a left-side row when no corresponding right-side row exists. `NOT EXISTS` is a common SQL pattern for it.

27. JOIN vs EXISTS — when should each be used?

Use a join when you need values from both table sources. Use `EXISTS` when you only need to know whether at least one related row exists.

28. Which physical join algorithms can SQL Server use?

The optimizer can use algorithms including Nested Loops, Merge Join, Hash Join, and in supported scenarios Adaptive Join.

29. Should developers force Nested Loops, Hash, or Merge Join?

Normally no. Let the optimizer choose unless execution-plan analysis and measured testing clearly show a reason to use hints. Microsoft warns that restrictive join hints can make plans worse.

30. What is join elimination?

Join elimination is an optimizer simplification where a logically unnecessary join may be removed when SQL Server can prove it doesn't affect the result. Trusted constraints such as foreign keys can provide information the optimizer uses for plan simplification.

J Final Quick Revision

EXAMPLE / FLOW

```
INNER JOIN
→ Matching rows
```

EXAMPLE / FLOW

```
LEFT JOIN
→ All left + matching right
```

EXAMPLE / FLOW

```
RIGHT JOIN
→ All right + matching left
```

EXAMPLE / FLOW

```
FULL OUTER JOIN
→ All matched and unmatched rows
```

EXAMPLE / FLOW

```
CROSS JOIN
→ Every possible combination
```

EXAMPLE / FLOW

```
SELF JOIN
→ Table joined to itself
```

EXAMPLE / FLOW

```
ON
→ Define relationship/match
```

EXAMPLE / FLOW

```
WHERE
→ Filter final rows
```

EXAMPLE / FLOW

```
CROSS APPLY
→ Correlated right source; unmatched left disappears
```

EXAMPLE / FLOW

```
OUTER APPLY
→ Correlated right source; unmatched left remains
```

EXAMPLE / FLOW

```
EXISTS
→ Does a match exist?
```

EXAMPLE / FLOW

```
NOT EXISTS
→ Does no match exist?
```

EXAMPLE / FLOW

```
Semi Join
→ Return left row if match exists
```

EXAMPLE / FLOW

```
Anti Join
→ Return left row if match doesn't exist
```

EXAMPLE / FLOW

```
Join performance
→ Indexes + statistics + cardinality + data types + query shape
```

The most important rule to remember is:

First understand the relationship and the result grain you need. Then choose the join type. Most join problems come from using the wrong relationship, misunderstanding one-to-many cardinality, or filtering an outer join in the wrong place.